KR MANGALAM UNIVERSITY

School of Engineering & Technology

Course Code & Name

ENSP465 — New Age Programming Languages Lab

Capstone Project — Phase I

Project Proposal & System Design

Project Title

Concurrent Web Scraper in Go

A goroutine-based worker-pool tool for fast, fault-tolerant extraction of data from many web pages

Field	Details
Student Names	Kartikay Gautam, N.Yugesh
Roll Numbers	2301010137,2301010199
Programme / Semester	B.Tech CSE — Semester VII
Academic Year	2026–27
Section	Section C
Submission Date	23rd October
Faculty Name	Dr. Hemraj Kumawat School of Engineering & Technology, KRMU

2. Problem Statement & Motivation

2.1 Problem Description

A large amount of useful information, such as product prices, news headlines, course listings, job postings and research abstracts, is published on web pages that offer no download button or API. People who need this information in a structured form (students, analysts, small businesses, researchers) end up copying it by hand from page to page. This is slow, error-prone and impossible to repeat regularly once the number of pages grows beyond a few dozen.

Automating the task with a simple script helps only partly. A script that downloads pages one after another spends almost all of its time idle, waiting for each remote server to respond. If each page takes about one second to arrive, collecting 500 pages takes more than eight minutes, even though the computer is doing almost nothing during that time. Naive scripts also tend to fail badly in real conditions: one slow or broken page can hang the whole run, one malformed page can crash the parser, and sending hundreds of requests at once to a single site can overload it or get the user blocked.

This project addresses that gap by building a small command-line tool that downloads many pages **concurrently**, extracts a defined set of fields from each, tolerates failures, limits its own request rate, and writes the results to a clean JSON or CSV file together with a short summary of what succeeded and what failed.

2.2 Real-World Context or Domain

The project belongs to the domain of **command-line data-collection utilities** (web data extraction and data-pipeline tooling). The target users are students, researchers, data analysts and small teams who want a lightweight, scriptable tool they can run from a terminal or a scheduled job, without installing a large framework or a browser.

2.3 Motivation

- **Real and common need:** web data extraction is a routine first step in data analysis, price comparison, content auditing and academic research [6][7].
- **Large, easily measured speed-up:** the work is dominated by network waiting, so running requests concurrently can reduce total time from minutes to seconds. This makes the benefit of the design directly measurable.

- **Reliability matters as much as speed:** a tool that survives timeouts, retries transient errors and reports failures clearly is more useful than a fast one that crashes.
- **Fits the course outcomes:** the problem naturally exercises Go's concurrency model (goroutines, channels, synchronisation) together with its standard HTTP, JSON and testing libraries.

2.4 Existing Tools and Their Limitations

Existing tool	What it does	Limitation for this use case
Scrapy (Python) [7]	Full-featured scraping framework with spiders, pipelines and scheduling.	Heavy to set up and learn; requires a Python environment; more than a small, one-purpose tool needs.
Colly (Go) [6]	Popular Go scraping library with callbacks and rate limiting.	A large third-party library that hides the concurrency; it does not demonstrate or let the user control the worker-pool design.
Python scripts (requests + BeautifulSoup)	Short scripts that fetch and parse pages.	Sequential by default, so slow; concurrency has to be added manually and carefully because of Python's threading model.
Browser extensions and online scrapers	Point-and-click extraction.	Manual, limited page counts, often paid, and hard to automate or embed in a script.
curl / wget in shell loops	Download raw pages.	No parsing, no structured output, limited error handling and no controlled parallelism.

None of these gives a **small, self-contained, single-binary tool** with transparent, configurable concurrency, built-in fault handling and structured output. That is the gap this project fills.

3. Proposed Solution & Language Justification

3.1 Project Overview

The Concurrent Web Scraper is a command-line program that reads a list of page URLs from a text file and downloads them in parallel using a configurable pool of worker goroutines. For each page it extracts the title, meta description, heading and link counts, HTTP status and response time, applying timeouts, retries and per-host rate limiting. The results are collected

through channels, saved as JSON or CSV, and summarised in the terminal (pages scraped, failures, total time).

3.2 Language Chosen

Go (Golang) is selected as the single implementation language.

3.3 Language Justification

1. **Goroutines make massive concurrency cheap.** Scraping is I/O-bound: each request mostly waits on the network. A goroutine starts with only a few kilobytes of stack and is scheduled by the Go runtime rather than the operating system, so the tool can run tens or hundreds of fetches at once with very little memory and without hand-written thread management [1][4].
2. **Channels give safe communication without shared state.** URLs flow to workers through a `jobs` channel, and extracted records flow back through a `results` channel. Because workers do not modify shared variables, whole classes of data races are avoided. This is Go's "share memory by communicating" principle and maps one-to-one onto the worker-pool design [1][4].
3. **context and sync.WaitGroup handle timeouts, cancellation and shutdown.** Every request can carry a deadline through `context.WithTimeout`, the whole run can be cancelled cleanly (for example on Ctrl+C), and a `WaitGroup` guarantees the results channel is closed only after all workers have finished [2][4].
4. **A complete standard library for the job.** `net/http` provides a production-quality HTTP client with timeouts, `encoding/json` and `encoding/csv` produce the output, and `flag` handles the command line, so only one small external package (the HTML tokenizer) is needed [3][5].
5. **Built-in correctness tooling.** `go test` with the race detector (`-race`) can automatically find data races in concurrent code, and `net/http/httptest` lets the scraper be tested against a local fake server, which makes speed benchmarks repeatable [8].
6. **Single static binary.** `go build` produces one executable with no runtime to install, which suits a command-line utility that is easy to demonstrate and distribute on Windows, Linux or macOS.

3.4 Key Libraries and Frameworks

Library / package	Purpose
`net/http`	HTTP client with timeouts; `httptest` for local test servers
`golang.org/x/net/html`	Tokenising and parsing HTML to extract title, meta tags, headings and links [5]
`context`, `sync`	Cancellation, deadlines, WaitGroup, mutex where needed
`golang.org/x/time/rate`	Per-host rate limiter to avoid overloading servers
`encoding/json`, `encoding/csv`	Writing results
`flag`	Command-line options (workers, timeout, retries, output format)
`testing`, race detector	Unit tests, benchmarks, data-race checks

4. Objectives & Scope

4.1 Objectives

1. Implement a **worker pool** in Go with a user-configurable size from 1 to 100 goroutines, fed by a jobs channel and drained by a results channel.
2. Achieve at least a **5× speed-up** over a sequential baseline when scraping 100 pages with 10 workers, measured with a benchmark against a local test server with a fixed 200 ms response delay.
3. Extract **six fields** from every successfully fetched page: URL, HTTP status code, title, meta description, number of links and number of headings, plus the response time in milliseconds.
4. Make the tool **fault-tolerant**: a default 10-second timeout per request, up to 2 retries with backoff for temporary errors, and guaranteed completion of the run even if some pages fail, with every failure recorded with a reason.
5. Export results as **JSON and CSV** with a terminal summary, and reach at least **70% test coverage** on core packages with all tests passing under `go test -race`.

4.2 In-Scope Features (Phases II and III)

- Reading URLs from a text file; validating and de-duplicating them.

- Concurrent fetching with a configurable worker count, timeout and retry limit.
- Per-host rate limiting and a clear User-Agent string.
- HTML parsing to extract the defined fields.
- Result collection through channels and output to JSON or CSV.
- Terminal summary: pages attempted, succeeded, failed (by reason), total time and average response time.
- Graceful shutdown on Ctrl+C using `context` cancellation.
- Unit tests, a benchmark comparing sequential and concurrent runs, and a README with usage examples.

4.3 Out of Scope

- Recursive crawling or following links across a whole website (only the URLs supplied are fetched).
- Executing JavaScript or scraping pages that render content in the browser (no headless browser).
- Logging in, cookies, CAPTCHA solving or any authentication.
- Parsing of `robots.txt` [9]. The tool is intended for pages the user is permitted to access, and the rate limit and identifiable User-Agent are provided for polite use.
- A graphical or web interface, database storage, distributed scraping across machines, and scheduled/continuous runs.

5. System Architecture & Design

5.1 High-Level Architecture

The system follows a **pipeline with a worker pool** structure. A single dispatcher feeds URLs into a channel, N identical worker goroutines pull jobs from it, fetch and parse each page, and push the resulting records into a second channel that a single collector reads. Figure 1 shows the modules and how they interact.

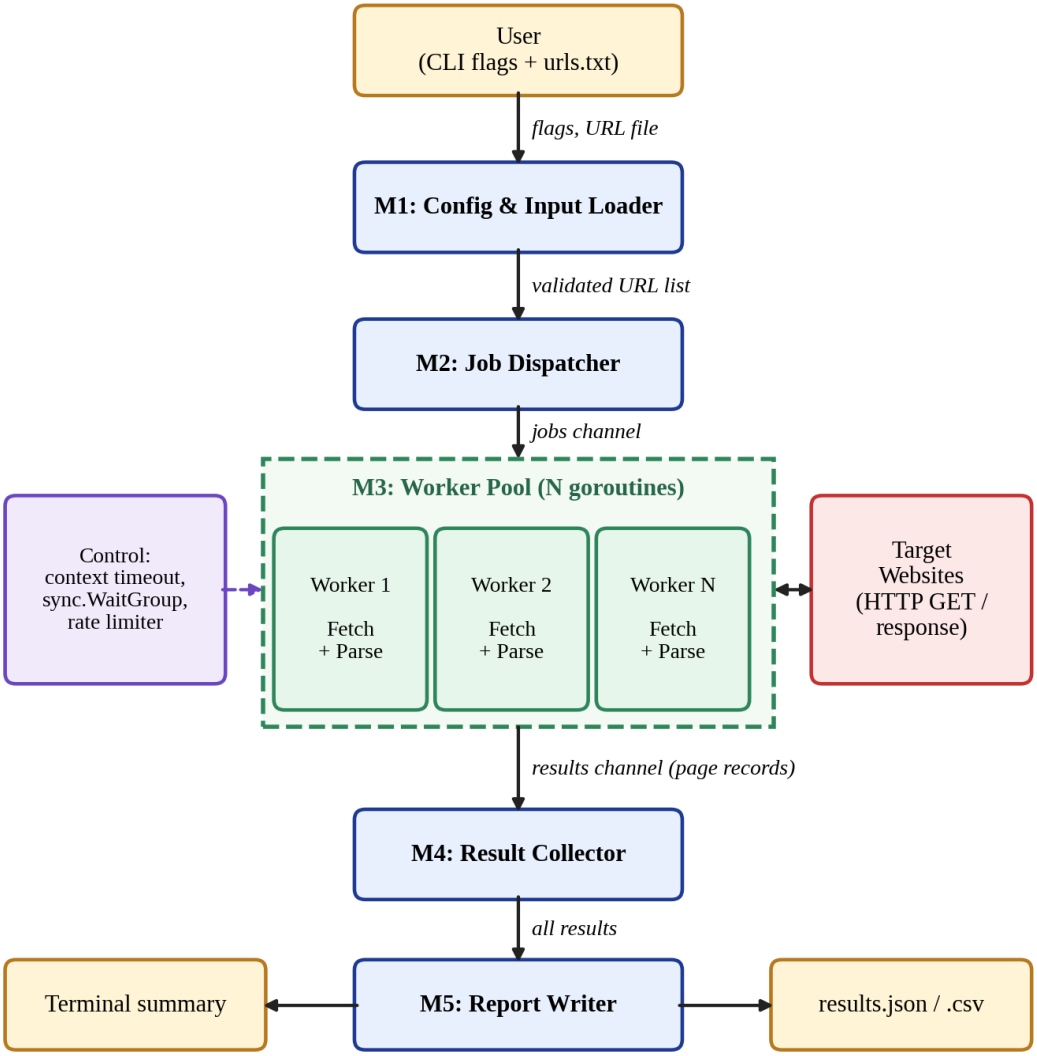


Figure 1: High-level architecture of the Concurrent Web Scraper

5.2 Component Description

M1 — Config & Input Loader. Reads the command-line flags (input file, number of workers, timeout, retries, rate limit, output path and format) and the URL file. It trims blanks, removes duplicates and rejects malformed URLs. *Input:* flags and a text file. *Output:* a validated `Config` struct and a slice of URLs.

M2 — Job Dispatcher. A single goroutine that wraps each URL in a `Job` value and sends it into the buffered `jobs` channel, stopping early if the context is cancelled, and closes the channel when all URLs are sent. *Input:* URL list. *Output:* a stream of `Job` values.

M3 — Worker Pool. N goroutines started together, each looping over the `jobs` channel. For every job a worker waits for the per-host rate limiter, performs an HTTP GET under a timeout (the **Fetcher**), retries temporary failures, and passes the response body to the **Parser**, which

uses the HTML tokenizer to extract the title, meta description, and counts of headings and links. *Input:* `Job`. *Output:* a `PageResult` (success or failure with reason) sent to the `results` channel.

M4 — Result Collector. The only goroutine that reads the `results` channel. It appends results to a slice and updates counters. Because only this goroutine touches the collected data, no mutex is needed on it. A helper goroutine closes `results` after the WaitGroup shows that all workers have exited. *Input:* `PageResult` stream. *Output:* the complete result set and run statistics.

M5 — Report Writer. Sorts results into the input order and writes them to a JSON or CSV file, then prints the terminal summary. *Input:* result set and statistics. *Output:* a result file and a console summary.

Control layer. A root `context` (with cancel on Ctrl+C), a `sync.WaitGroup` and a per-host rate limiter are shared by the modules above to coordinate shutdown and politeness.

5.3 Data Flow Diagram (Level 1)

Figure 2 shows the main scraping workflow as a Level 1 data flow diagram: data enters from the user and the URL file, passes through six processes, and leaves as a result file and a terminal summary.

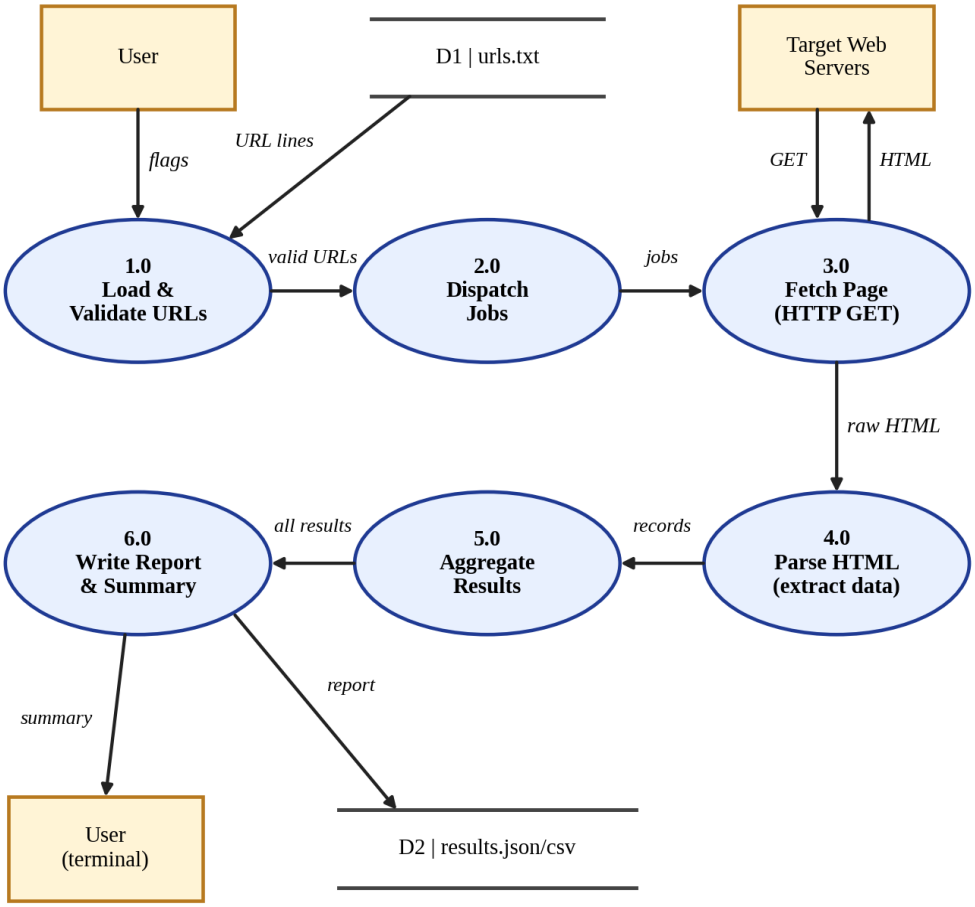


Figure 2: Data Flow Diagram (Level 1) of the main scraping workflow

5.4 Input–Process–Output (IPO)

Stage	Description	Details
Input	What the user provides	A text file with one URL per line; flags such as `-workers 10 -timeout 10s -retries 2 -format json -out results.json`.
Process	What the system does	Validate and de-duplicate URLs, dispatch jobs, fetch pages concurrently under timeouts and rate limits, retry temporary failures, parse HTML, collect results through a channel, aggregate statistics.

Stage	Description	Details
Output	What the user receives	A JSON or CSV file with one record per URL (including failures and their reasons) and a terminal summary of totals, failures by category and elapsed time.

5.5 Key Data Structures and Go Features

The core types are shown below. Channels carry values of these types between goroutines.

```
type Job struct {
    Index   int      // original position, for ordered output
    URL     string
    Attempt int
}

type PageResult struct {
    Index        int    `json:"-"`
    URL          string `json:"url"`
    StatusCode   int    `json:"status"`
    Title        string `json:"title"`
    MetaDesc     string `json:"meta_description"`
    LinkCount    int    `json:"links"`
    HeadingCount int    `json:"headings"`
    ResponseMs   int64  `json:"response_ms"`
    Error        string `json:"error,omitempty"`
}
```

- **Channels:** `jobs chan Job` (buffered) and `results chan PageResult`, as the only link between dispatcher, workers and collector.
- **Goroutines and sync.WaitGroup:** N workers plus dispatcher and closer goroutines; the WaitGroup ensures `results` is closed only after every worker returns.
- **context.Context:** per-request timeouts and run-wide cancellation, checked with `select` in dispatcher and workers.
- **Interfaces:** small `Fetcher` and `Parser` interfaces so that tests can substitute fake implementations.
- **Struct tags and error wrapping:** struct tags drive JSON output; wrapped errors (`fmt.Errorf("...: %w", err)`) allow failures to be classified (timeout, DNS, HTTP 4xx/5xx, parse error).
- **Testing features:** table-driven tests, `httptest.Server`, benchmarks, and the race detector.

6. References

[1] A. A. A. Donovan and B. W. Kernighan, The Go Programming Language. Boston, MA, USA: Addison-Wesley, 2015.

[2] The Go Authors, "Package context," Go Packages. [Online]. Available: https://pkg.go.dev/context. Accessed: Oct. 2026.

[3] The Go Authors, "Package net/http," Go Packages. [Online]. Available: https://pkg.go.dev/net/http. Accessed: Oct. 2026.

[4] S. Ajmani, "Go Concurrency Patterns: Pipelines and cancellation," The Go Blog, Mar. 2014. [Online]. Available: https://go.dev/blog/pipelines. Accessed: Oct. 2026.

[5] The Go Authors, "Package golang.org/x/net/html," Go Packages. [Online]. Available: https://pkg.go.dev/golang.org/x/net/html. Accessed: Oct. 2026.

[6] "gocolly/colly: Elegant Scraper and Crawler Framework for Golang," GitHub. [Online]. Available: https://github.com/gocolly/colly. Accessed: Oct. 2026.

[7] Scrapy Developers, "Scrapy 2.x Documentation." [Online]. Available: https://docs.scrapy.org/. Accessed: Oct. 2026.

[8] The Go Authors, "Data Race Detector," go.dev. [Online]. Available: https://go.dev/doc/articles/race_detector. Accessed: Oct. 2026.

[9] M. Koster, G. Illyes, H. Zeller, and L. Sassman, "Robots Exclusion Protocol," IETF, RFC 9309, Sep. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9309. Accessed: Oct. 2026.